

git Basics

Distributed version control

Activity: Getting Started

1. Log in with an account at github.com
2. Visit <https://github.com/thedataincubator/devsecops>
3. Click *Fork* button in top left
4. Check options; click *Create fork*
5. Let Github ponder this for a while, until it shows your fork
6. Click *Code* › *Codespaces* › + to create new Codespace

It will take several minutes to launch your code space

Version Control Systems (VCS)

Keep track of history of code

- Determine when and why (hopefully) change was made
- Understand when vulnerability or bug introduced
- Roll back to previous states

Version Control Enables Collaboration

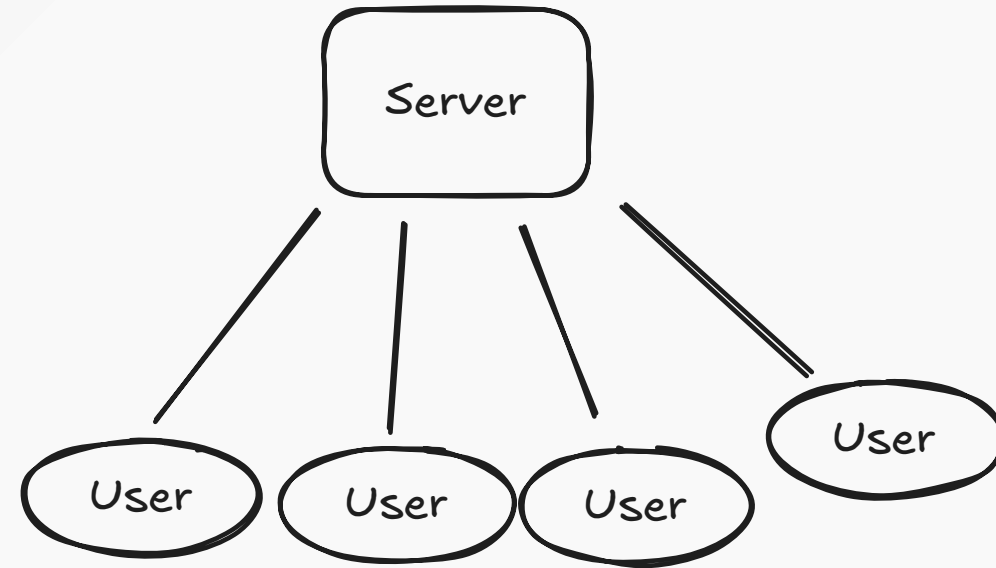
Shared state of system

Mechanism to exchange changes

Tools to resolve conflicting changes

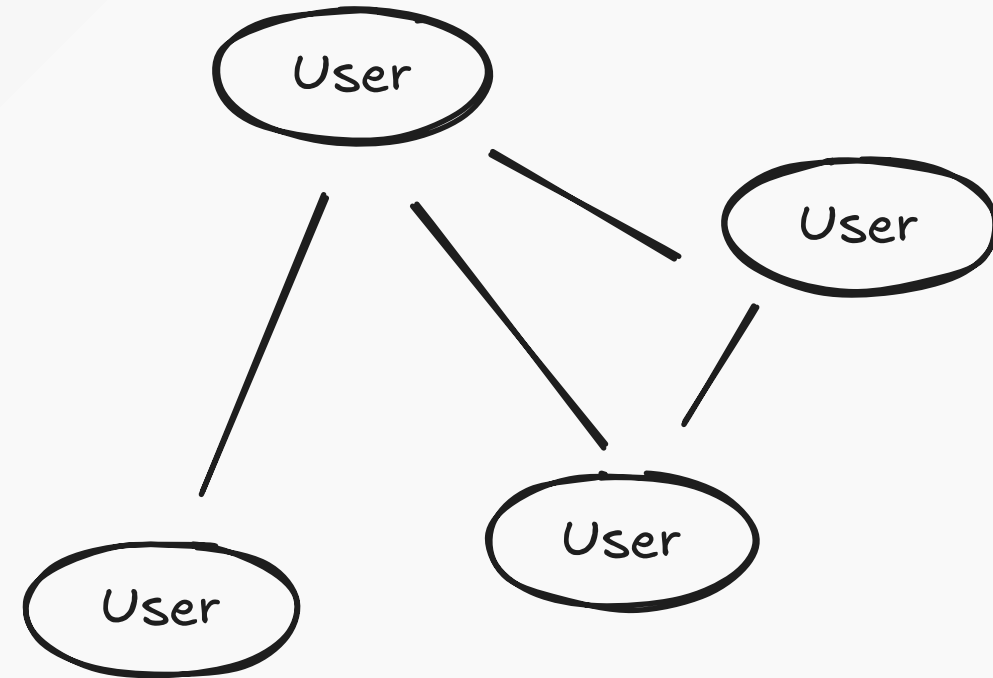
Early VCS: Client/Server

- Server contains official state of project
- All users check out, check in with central server
- Merges difficult; generally avoided
- *RCS, CVS, Subversion*



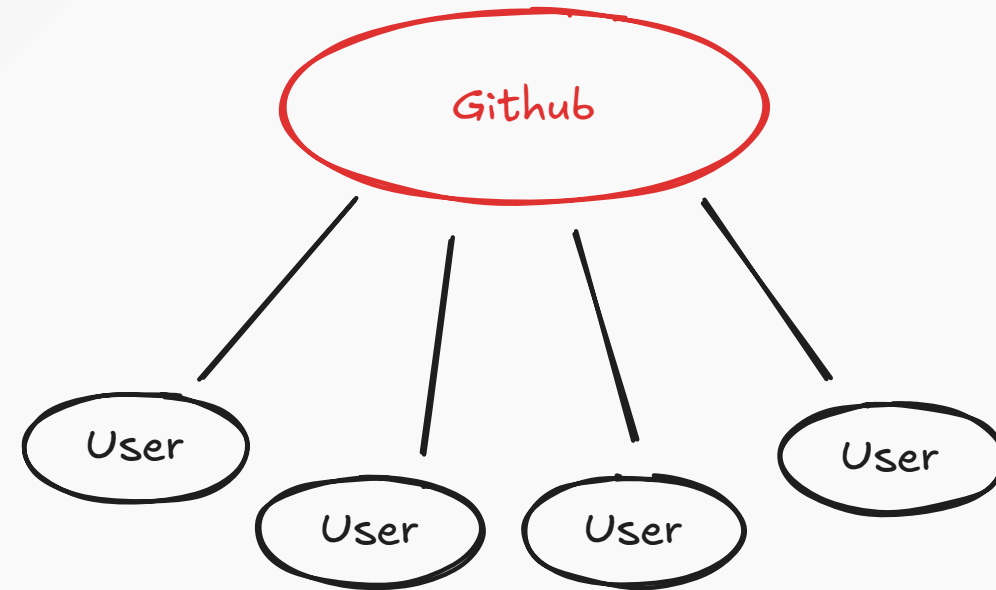
Distributed VCS (DVCS)

- All users have full history
- Any user can exchange data with any other user
- "Official" version is a social construct
- Early 2000s: *BitKeeper*, *Mercurial*, *bazaar*, *Fossil*
- *git* developed by Linus Torvalds, wins most mindshare



DVCS in Practice

- One user is treated as official
- All other users pull and push changes to that one user
- Role often filled by *Github*, *Gitlab*
 - Generally provide other DevOps tooling



Codespaces Overview

Virtual development machines running on Github servers

Interface based on VSCode

Machine configured according to `.devcontainer` folder

Can create multiple codespaces per repo

Codespaces shutdown after 30 minutes; restart quickly with preserved filesystem state

Codespaces Quotas

Monthly quota for free GitHub accounts:

Compute: 120 CPU-hours

- Default machine has 2 CPUs
- Only charged while Codespace running

Storage: 15 GB-months

- Based on storage used in each Codespace
- Usage accrues for running or stopped Codespaces

Managing Codespaces

View and manage your Codespaces at github.com/codespaces

Track your usage at github.com/settings/billing

Suggestions:

- *Run only one Codespace at a time*
- *Delete Codespace if you don't plan to use it for some time*
 - *We'll talk about what to do to preserve configuration*

VSCode AI Suggestions

If you don't want them:

-  *File* *Preferences* *Settings*
- Three tiers of settings
 - *User*: All Codespaces you open
 - *Remote*: This particular Codespace
 - *Workspace*: This particular repo
- Search for "Inline Suggest: Enabled"
 - Toggle off

Getting Around git

We'll be primarily using the command line interface

- Always available
- Always capable

Many graphical interfaces, including in VSCode

- Interface to same operations
- Use them if you like!

Initializing a git Repo

```
git init directory
```

Creates hidden `.git` folder in `directory`

git will use this directory to track the state and history of this repository

Current Status

```
git status
```

Shows current branch, current state

An alias I use often:

```
alias gits="git status"
```

git's Two-Stage Commit Process

```
git add file.txt
```

Adds changes to the **staging area**

⇒ Lets us prep changes from several files into a single commit

```
git commit
```

Saves all changes in the staging area

Prompts for commit message

Here, VSCode; generally terminal text editor (*vi*, *nano*)

Good Commit Messages

Commit message should say *what* and *why*, not *how*

Short summary of the changes in this commit

This is followed by one or more paragraphs describing what was done and why this change was made. We don't need to go into details of how we accomplished the change; we can derive that from the git history.

Old timers (like me) try to keep the first line shorter than 60 characters. The paragraphs should be hard-wrapped at 72 characters. This helps the commit messages look good in various tools.

Good Commit Messages

- First lines often written in "imperative" mood
Add new file, not *Adds new file*
 - This matches style of automatic merge commits
- Start with capital letter
- No period at end of first line
- Some teams add commit **type** or **scope** to beginning
 - Types include feature, bugfix, docs, *etc.*
 - Scope indicates portion of project affected

Opinions on the Staging Area

- Some dislike the extra steps

```
git commit -a
```

Commits all *tracked* files.

- Some like the ability to construct logical commits

```
git add -p file.txt
```

Lets you add individual "chunks" of a file

⇒ Split mixed ideas into two commits

Examining History

See past commits: `git log`

Compact log: `alias gl="git log --all --decorate --oneline --graph"`

Compare commits: `git diff id1 id2`

- Identifiers can be
 - hashes
 - branch names
 - `HEAD` for current commit
 - `HEAD^` for parent of current commit

Branches

Branches allow switching between several versions

- Create a new branch: `git checkout -b my-branch`
- List all branches: `git branch`
- Switch current branch: `git checkout this-branch`

Switching branches changes the contents of the *working directory*, but all history is saved in `.git` folder

Default Branch: *main* or *master*

The default branch used to be called *master*

Recently, orgs have been shifting to calling it *main*

The names don't matter to git

You can type *ma<tab>* in either case!

Merging

```
git merge other-branch
```

Brings all changes committed to *other-branch* into the current branch

If there are no **conflicts**, completes automatically

Merge Conflicts

If both branches have changes in the same areas, git doesn't know which to keep. It makes you figure it out.

```
This line is in both branches
<<<<<< HEAD
This line is in the current branch
=====
This line is the incoming branch
>>>>>> branch
```

You must edit the file to resolve the conflict

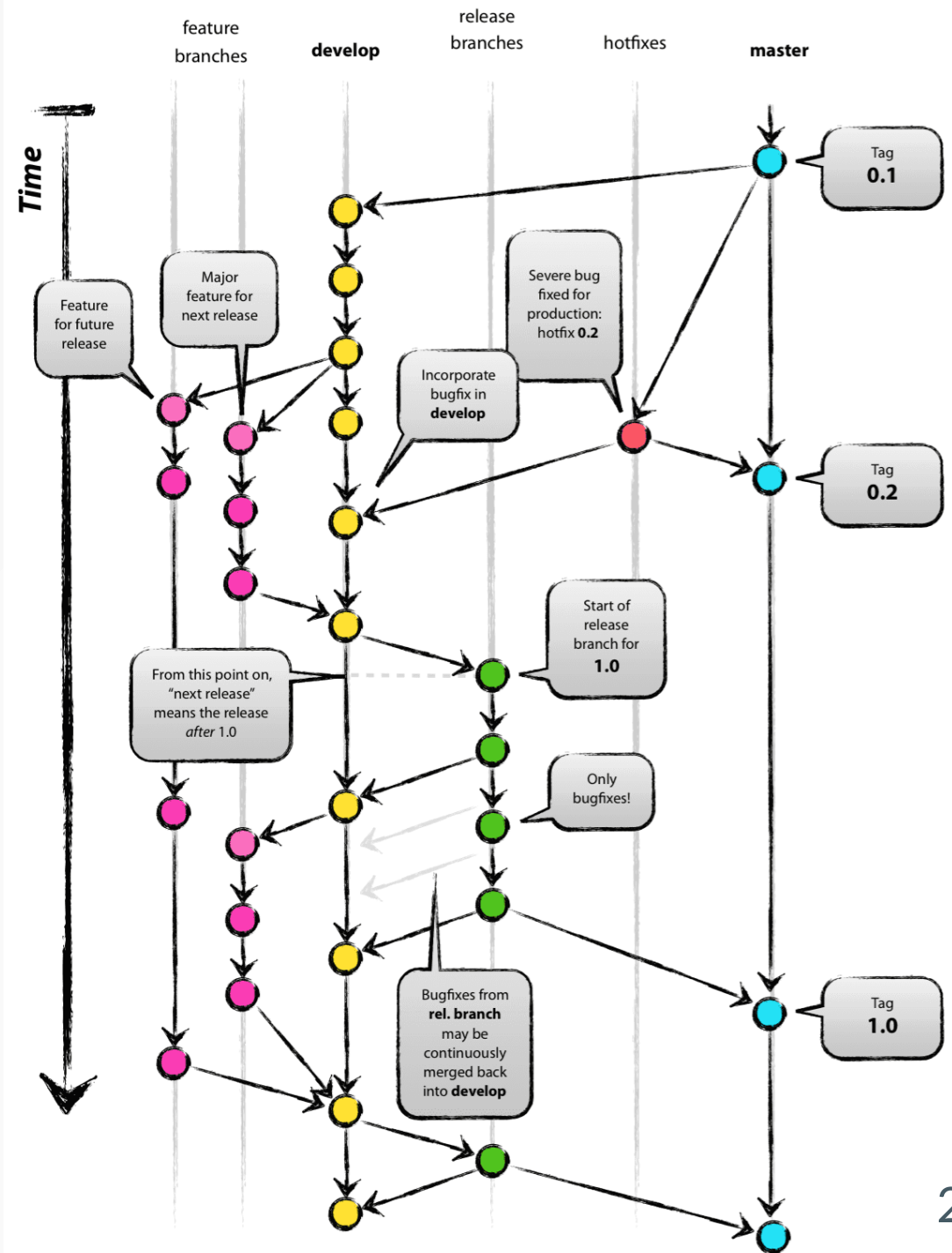
Then, add and commit

git-flow Strategy

- *main* branch for published
- *develop* for primary work
- *release* branches to prep for release into *main*
- *feature* and *hotfix* branches

From Vincent Driessen (2010)

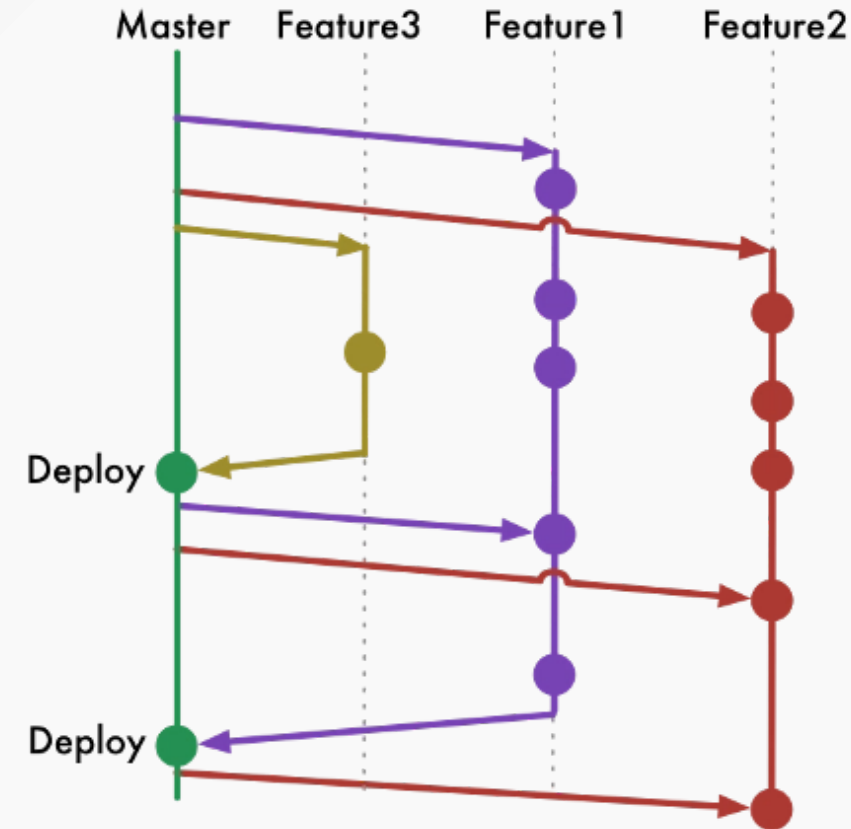
Not for agile development



Trunk-Based Development

- *main* keeps up with development
- Short lived *feature* or *bugfix* branches
 - Merged into *main* once passing tests
- *main* should always be passing CI
 - Able to deploy after any merge

Most DevOps teams use a pattern similar to this



Activity: Editing README

1. Create a new branch named *readme-link*
2. Add this line to the beginning of README.md

```
[Launch Codespaces](https://codespaces.new/username/devsecops?quickstart=1)
```

Replace `username` with your Github username

3. Commit the changes

Copying Repos

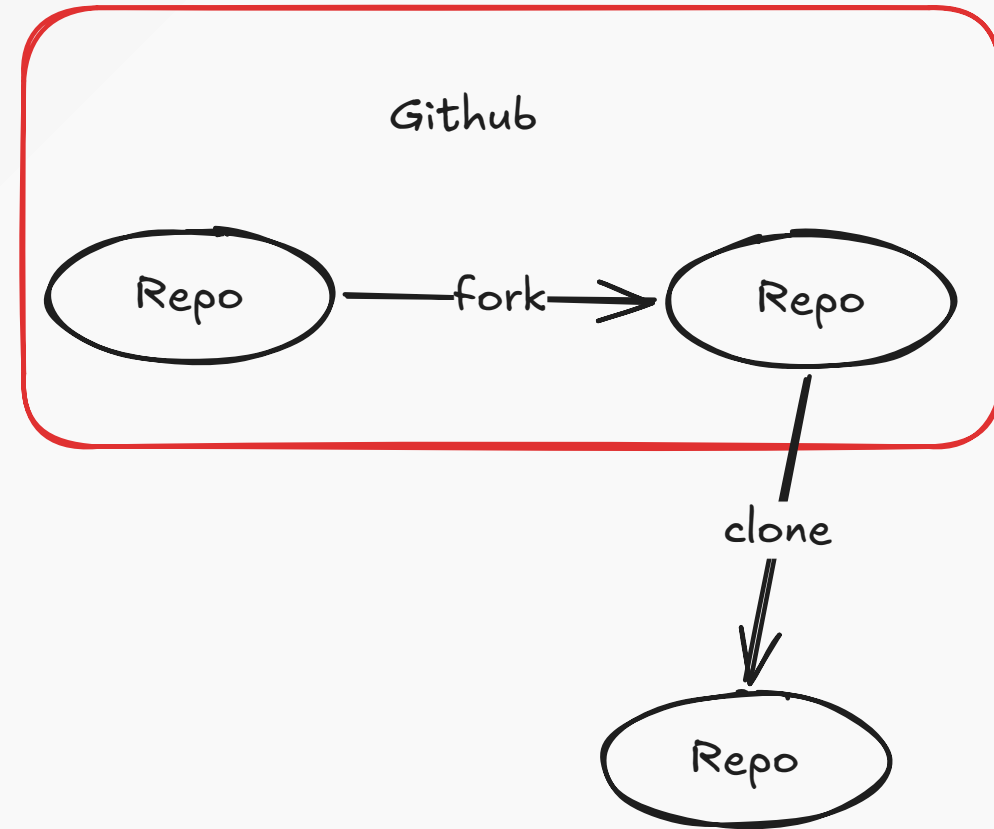
Fork: Make a copy of a repo on the same host

Clone: Make a local copy of a repo

```
git clone repository-location
```

Access via

- SSH: Requires public key on host
- HTTPS: Sign in with password



git Remotes

Remotes are other repos that your repo knows about

```
git remote          # Lists names
git remote -v       # Shows connection string
git remote show origin # Detailed information on branches
```

The source of a clone is conventionally named *origin*.

Communicating with Remotes

`git fetch [remote]` — Find out what's new on the remote

`git checkout branch-name` — Creates local branch with same name that **tracks** remote branch (use `origin/branch-name` to be explicit)

`git pull` — Gather changes from the remote branch being tracked

`git push -u origin branch-name` — Create new branch on *origin* and set up tracking (only needed first time)

`git push` — Send changes to the remote tracking branch

Activity: Push to Github

1. Push your *readme-link* branch to your *origin*

A notification should appear on the main Github page for your repo

Pull Requests

A common approach to approving changes

1. Open PR, request reviewer
2. Get feedback from reviewer
3. Make changes to address comments
 - Usually additional commits to same branch
4. Merge when approved
 - CLI or web UI

Not part of git itself; popularized by GitHub

Activity: Pull Request

1. Create a PR for your *readme-link* branch
2. Approve the PR
(if it meets your standards)
3. Merge the *readme-link* branch into *main*, via the web UI
4. In your Codespace, checkout the *main* branch
5. Pull changes to the *main* branch

Your edits to the README should now be in *main* locally

Multiple Remotes

A repo can have multiple remotes

We will publish updates to *thedataincubator/devsecops*

⇒ Let's add that as another remote

```
git remote add upstream https://github.com/thedataincubator/devsecops.git
```

Name *upstream* is arbitrary

```
git fetch upstream # Gather info about branches in this remote
```

Activity: Find the Secret Word

In the *thedataincubator/devsecops* repository, there is a branch named *git-exercise*

This branch has the secret word of the day in

`exercises/git/README.md`

Find out what it is!